



MOHAMMED V UNIVERSITY
FACULTY OF SCIENCES

MASTER IGOV

Lab 2
Consensus Mechanisms

BLOCKCHAIN: FOUNDATIONS & APPLICATIONS

Written by:
Hajar CHAHBI

Supervised by:
Pr. YAHYA BENKAOUZ

2025–2026

Contents

1	Objectives of the Lab	2
2	Part 1 — Proof of Work (PoW)	2
2.1	Principle	2
2.2	Implementation (Java)	2
2.3	Experimentation	2
2.4	Plots	2
3	Part 2 — Basic Paxos (Banking System)	4
3.1	Model	4
3.2	Initial Log State	4
3.3	Paxos Execution Flow	4
3.4	Lagging Server (Lab Hint)	4
3.5	Execution Example	5
4	Execution	5
4.1	Compilation	5
4.2	PoW	5
4.3	Paxos	6
5	Conclusion	6

List of Figures

1	PoW — Threshold vs Execution Time	3
2	PoW — Threshold vs Number of Attempts	3
3	Paxos execution example for a Withdraw request	5

1 Objectives of the Lab

This lab has two main objectives:

- To implement a lightweight version of the Proof of Work (PoW) mechanism based on SHA-256, where the difficulty is defined by the number of leading zeros in the hexadecimal hash.
- To implement a Basic Paxos protocol to replicate a transaction log across three banking servers (A, B, C), where each server plays the roles of proposer, acceptor, and learner.

2 Part 1 — Proof of Work (PoW)

2.1 Principle

The goal is to find a nonce (32-bit) such that:

$$\text{SHA256}(\text{message} \parallel \text{nonce})$$

produces a hash whose hexadecimal representation starts with **k leading zeros**, where k is the required threshold.

2.2 Implementation (Java)

The Proof of Work mechanism was implemented in Java. A main method was developed to automatically test the PoW algorithm on several randomly generated messages of different sizes and for different threshold values. For each execution, the program records the number of attempts required to find a valid nonce as well as the total execution time.

2.3 Experimentation

For each threshold, the following metrics are measured:

- the number of attempts required;
- the execution time.

The results are exported to a CSV file (`pow_results.csv`).

2.4 Plots

The required plots are:

- Threshold vs execution time
- Threshold vs number of attempts

The plots were generated using a Python script based on the experimental results produced by the Java implementation.

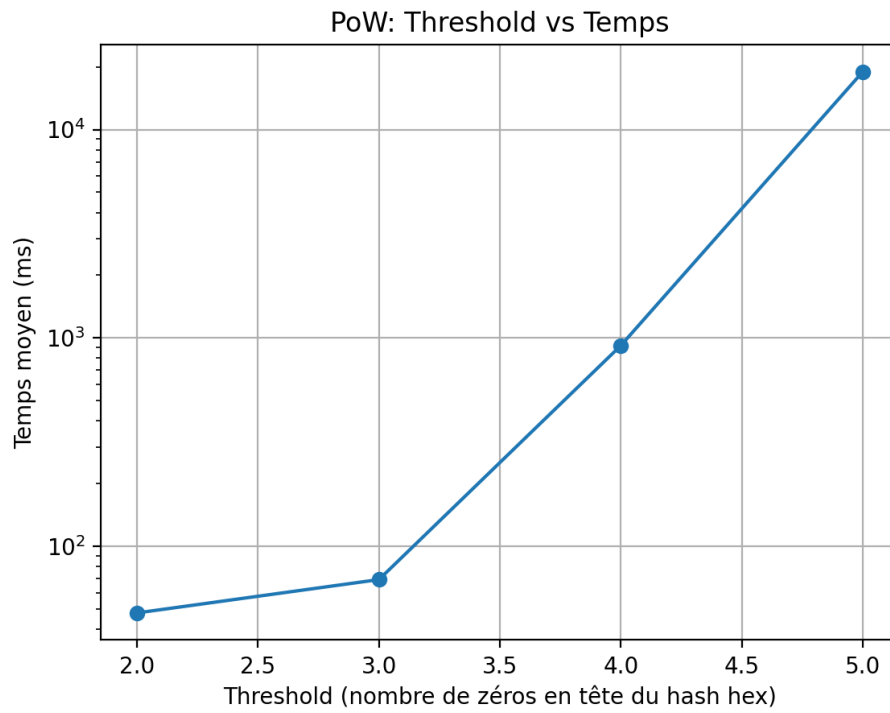


Figure 1: PoW — Threshold vs Execution Time

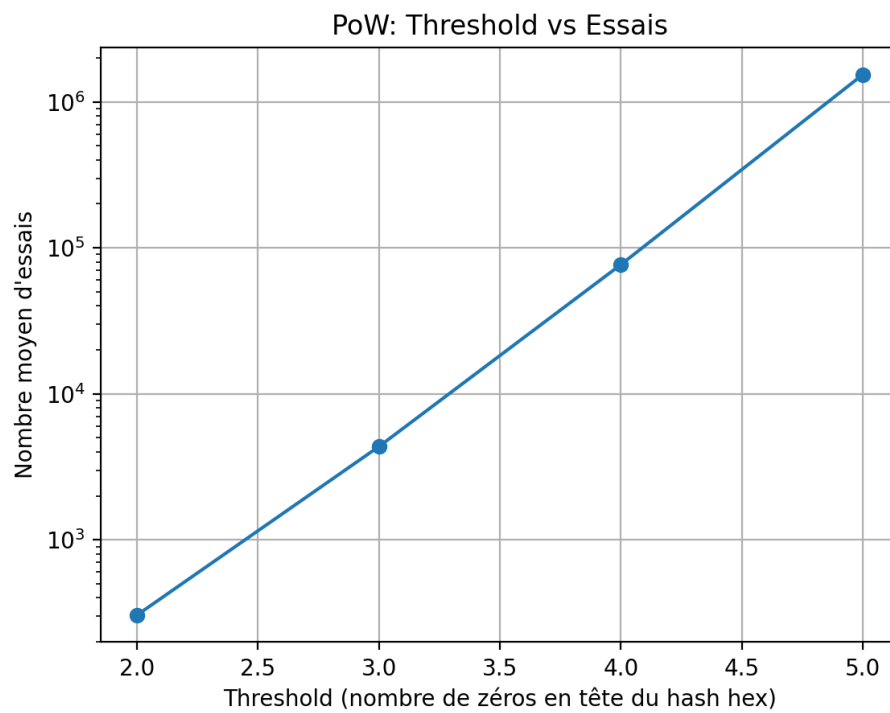


Figure 2: PoW — Threshold vs Number of Attempts

Analyse : The number of attempts grows approximately as 16^k , which corresponds to the probability of obtaining k consecutive hexadecimal zeros in a SHA-256 hash.

This behavior is explained by the fact that each hexadecimal digit has 16 possible values. Therefore, the probability of obtaining a hash with k leading hexadecimal zeros is approximately $(1/16)^k$, which leads to an expected number of attempts proportional to 16^k .

Although the vertical axis of the plots is displayed using a base-10 logarithmic scale for readability, the observed growth remains consistent with an exponential behavior proportional to 16^k .

3 Part 2 — Basic Paxos (Banking System)

3.1 Model

We simulate three servers A, B, and C (replicas), and two clients (1 and 2). Each **deposit** or **withdraw** request can be sent to any server. The server receiving the request becomes the proposer and launches a Paxos instance for the next log position. Each client transaction triggers a new Paxos consensus instance. The value agreed upon by the majority of servers is then stored in the next available position of the replicated log.

3.2 Initial Log State

Each server starts with the same replicated log (as specified in the lab description):

- pos 0: \$100
- pos 1: \$150
- pos 2: \$130

3.3 Paxos Execution Flow

For a given log position:

1. **Prepare:** the proposer sends **PREPARE(n)** to the acceptors.
2. **Promise:** if a majority (2/3) promises, the protocol continues.
3. **Accept:** the proposer sends **ACCEPT(n, value)**.
4. **Majority:** if a majority accepts, the value is chosen.
5. **Learn/Commit:** the value is written to the log and learned by all servers.

3.4 Lagging Server (Lab Hint)

The system simulates a lagging server that may miss commit messages. When it later attempts to write to the log, it learns the chosen value from the majority of replicas.

3.5 Execution Example

The following figure illustrates an example execution of the Paxos protocol. A **Withdraw 30** request sent to server B triggers a Paxos instance for the next log position. The proposal is accepted by a majority of servers (2 out of 3), after which the chosen value is committed and replicated across all servers.

```
C:\Users\hajar\Downloads\consensus_lab_java>java -cp out Main paxos
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
=== Basic Paxos Bank Simulation ===
Servers: A, B, C (each acts as proposer/acceptor/learner)
Clients: 1, 2
Initial log: pos0=$100, pos1=$150, pos2=$130

Menu:
1) Deposit
2) Withdraw
3) Toggle lagging server (simulate late catch-up)
4) Show replica logs
0) Exit
> 2
Client id (1 or 2): 1
Send request to server (A/B/C): B
Amount: 30

Client 1 -> Withdraw 30 via Server B

Paxos started by B for log position 3 with proposal n=1066
  Prepare -> A : PROMISE (promisedN=1066)
  Prepare -> B : PROMISE (promisedN=1066)
  Prepare -> C : PROMISE (promisedN=1066)
  Phase 2 value selected: 100 (client value)
  Accept -> A : ACCEPT
  Accept -> B : ACCEPT
  Accept -> C : ACCEPT
Consensus reached for pos 3: chosen value = $100
Transaction committed at log position 3 with balance $100

>< 0 0
```

Figure 3: Paxos execution example for a Withdraw request

4 Execution

4.1 Compilation

```
javac -d out src/*.java
```

4.2 PoW

```
java -cp out Main pow
```

4.3 Paxos

```
java -cp out Main paxos
```

5 Conclusion

The Proof of Work implementation demonstrates that increasing the difficulty threshold leads to a rapid growth in both the number of attempts and the execution time, highlighting the computational cost required to satisfy the PoW condition.

The Paxos implementation illustrates how majority-based consensus ensures consistency in a replicated transaction log. It also shows that a lagging server can safely recover the correct state by learning the chosen value from the majority of replicas. om the majority.